

Number Theory

CodeIIEST Summer Bootcamp 2026

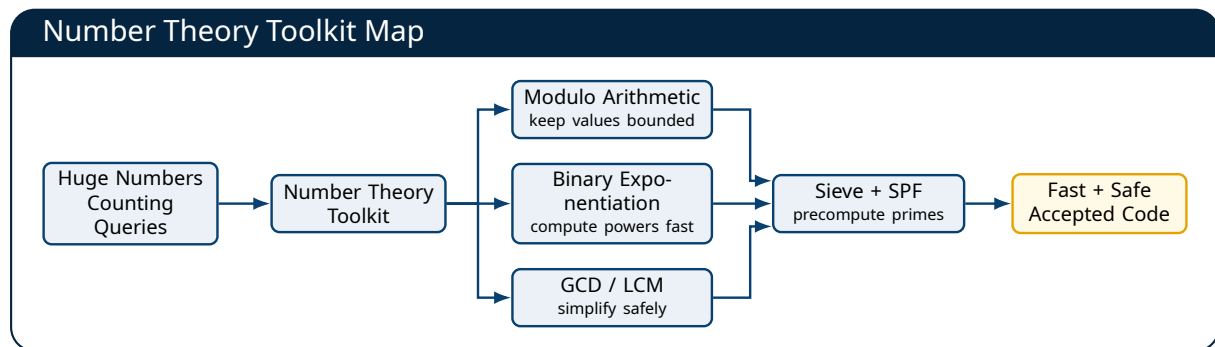
Week 06 · Prerequisites

Contents

1	Introduction	2
2	Modular Arithmetic	2
2.1	Properties of Modulo	3
3	Binary Exponentiation (Fast Pow)	4
4	Euclidean Algorithm (GCD & LCM)	5
5	Sieve of Eratosthenes & Precomputation	6
6	Fast Prime Factorization using SPF	7
7	Time Complexity Reference	9
8	Recommended Resources	9

1. Introduction

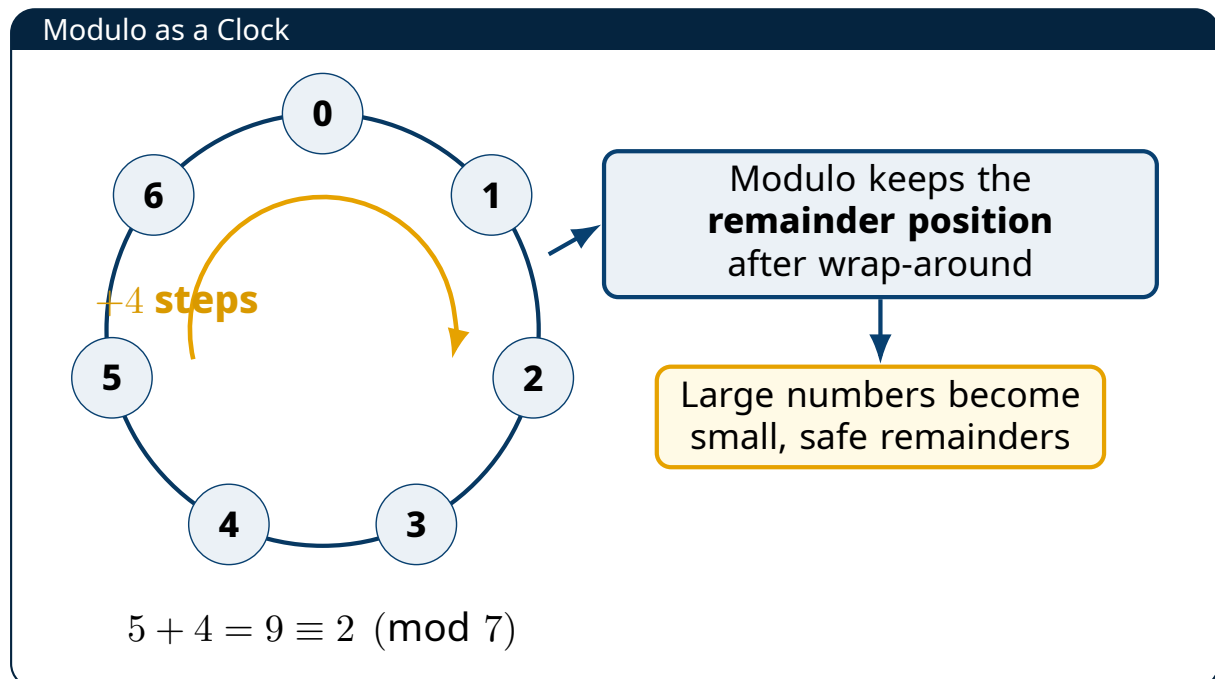
Number Theory is the backbone of many advanced competitive programming problems, especially in combinatorics, counting, and cryptography. Since numbers can grow astronomically large (exceeding standard 64-bit integer limits), mastering how to compute results under a modulo, find prime factors efficiently, and calculate large powers is absolutely crucial to avoiding Overflow Errors and Time Limit Exceeded (TLE) verdicts.



2. Modular Arithmetic

Concept

In competitive programming, the answer to counting problems is often extremely large. To prevent integer overflow, problems will ask you to output the answer "modulo $10^9 + 7$ " or "modulo 998244353". These specific numbers are chosen because they are large primes and fit perfectly within a 32-bit integer.

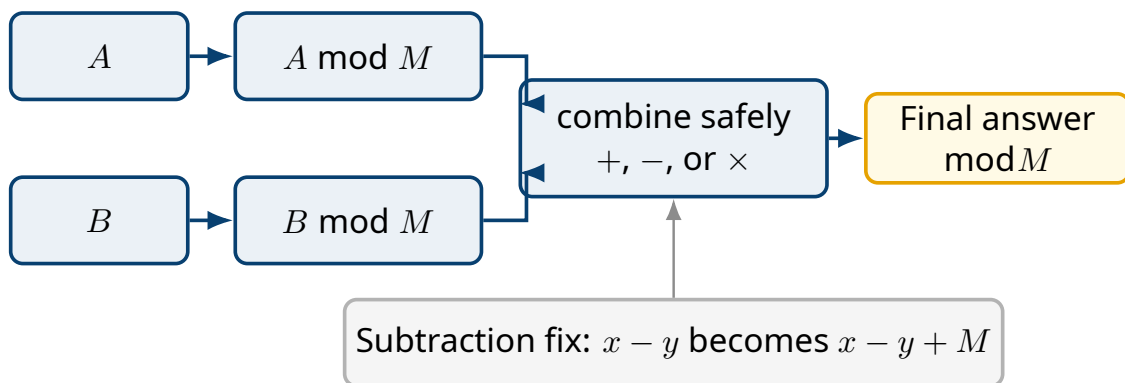


2.1 Properties of Modulo

Let M be the modulo (e.g., $M = 10^9 + 7$). When adding, subtracting, or multiplying, you must apply the modulo operator % at **every step**, not just at the end.

- **Addition:** $(A + B) \pmod{M} = ((A \pmod{M}) + (B \pmod{M})) \pmod{M}$
- **Multiplication:** $(A \cdot B) \pmod{M} = ((A \pmod{M}) \cdot (B \pmod{M})) \pmod{M}$
- **Subtraction:** $(A - B) \pmod{M} = ((A \pmod{M}) - (B \pmod{M}) + M) \pmod{M}$
(We add M before the final modulo to ensure the result is strictly positive).

Safe Modular Operation Pipeline



What about Division?

You **cannot** simply do $(A/B) \pmod{M}$. Instead, you must multiply by the Modular Multiplicative Inverse of B . That is: $(A \cdot B^{-1}) \pmod{M}$. We find B^{-1} using Fermat's Little Theorem and Binary Exponentiation.

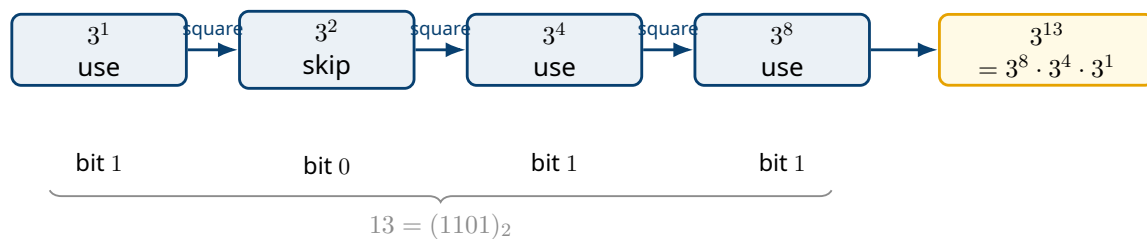
3. Binary Exponentiation (Fast Pow)

Concept

Binary Exponentiation is a technique to calculate $A^B \pmod M$ in $O(\log B)$ time instead of the naive $O(B)$ time.

Instead of multiplying A by itself B times, we repeatedly square the base and halve the exponent. For example, $3^{13} = 3^8 \cdot 3^4 \cdot 3^1$.

Binary Exponentiation Power Ladder



```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int MOD = 1e9 + 7;
5
6 long long binpow(long long a, long long b) {
7     long long res = 1;
8     a %= MOD; // Important if 'a' is already larger than MOD
9
10    while (b > 0) {
11        // If the current exponent bit is 1, multiply the result by 'a'
12        if (b % 2 == 1) {
13            res = (res * a) % MOD;
14        }
15        // Square the base and halve the exponent
16        a = (a * a) % MOD;
17        b /= 2;
18    }
19    return res;
20 }
```

4. Euclidean Algorithm (GCD & LCM)

Concept

The Euclidean Algorithm is used to find the Greatest Common Divisor (GCD) of two numbers efficiently in $O(\log(\min(A, B)))$ time. It relies on the principle that $\gcd(a, b) = \gcd(b, a \bmod b)$.

Euclidean Algorithm Remainder Chain



$$\text{lcm}(a, b) = \frac{a}{\gcd(a, b)} \cdot b$$

Divide first
to reduce overflow

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // Recursive GCD function
5 long long gcd(long long a, long long b) {
6     if (b == 0) return a;
7     return gcd(b, a % b);
8 }
9
10 // Function to find Least Common Multiple (LCM)
11 long long lcm(long long a, long long b) {
12     // Divide first to prevent overflow: (a * b) / gcd
13     return (a / gcd(a, b)) * b;
14 }
15
16 int main() {
17     // Note: C++17 has a built-in std::gcd(a, b) in the <numeric> header
18     !
19     cout << std::gcd(12, 18) << "\n"; // Outputs 6
20     return 0;
  
```

5. Sieve of Eratosthenes & Precomputation

Concept

When a problem requires answering multiple queries about primes, checking primes one by one is too slow. The **Sieve of Eratosthenes** precomputes all prime numbers up to N in $O(N \log \log N)$ time.

We create a boolean array representing all numbers up to N . We iterate from 2 to $\sqrt{N}$. If a number is marked as prime, we mark all of its multiples as composite (not prime).

Sieve of Eratosthenes: Mark Multiples

Blue cells are primes; red crosses are composites removed by the sieve

0	1	2	3	4	5	6	7	8	9
10	11	12	13	14	15	16	17	18	19
20	21	22	23	24	25	26	27	28	29
30	31	32	33	34	35	36	37	38	39

$p = 2$: start at 4

$p = 3$: start at 9

$p = 5$: start at 25

Start from p^2 : smaller multiples were already removed by smaller prime factors.

```

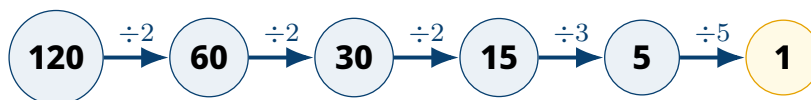
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int N = 1000000;
5 vector<bool> is_prime(N + 1, true);
6
7 void sieve() {
8     is_prime[0] = is_prime[1] = false; // 0 and 1 are not primes
9
10    for (int i = 2; i * i <= N; i++) {
11        if (is_prime[i]) {
12            // Mark all multiples of i as false, starting from i^2
13            for (int j = i * i; j <= N; j += i) {
14                is_prime[j] = false;
15            }
16        }
17    }
18 }
```

6. Fast Prime Factorization using SPF

Concept

While finding prime factors of a single number takes $O(\sqrt{X})$, finding them for Q queries will take $O(Q\sqrt{X})$, which causes TLE. We can optimize this to $O(\log X)$ **per query** by precomputing the **Smallest Prime Factor (SPF)** for all numbers up to N using a modified Sieve.

SPF Factorization Flow for 120



Output factors: 2, 2, 2, 3, 5

Each step uses
spf[x]

x	2	3	4	5	6	7	8	9	10	11	12	15
spf[x]	2	3	2	5	2	7	2	3	2	11	2	3

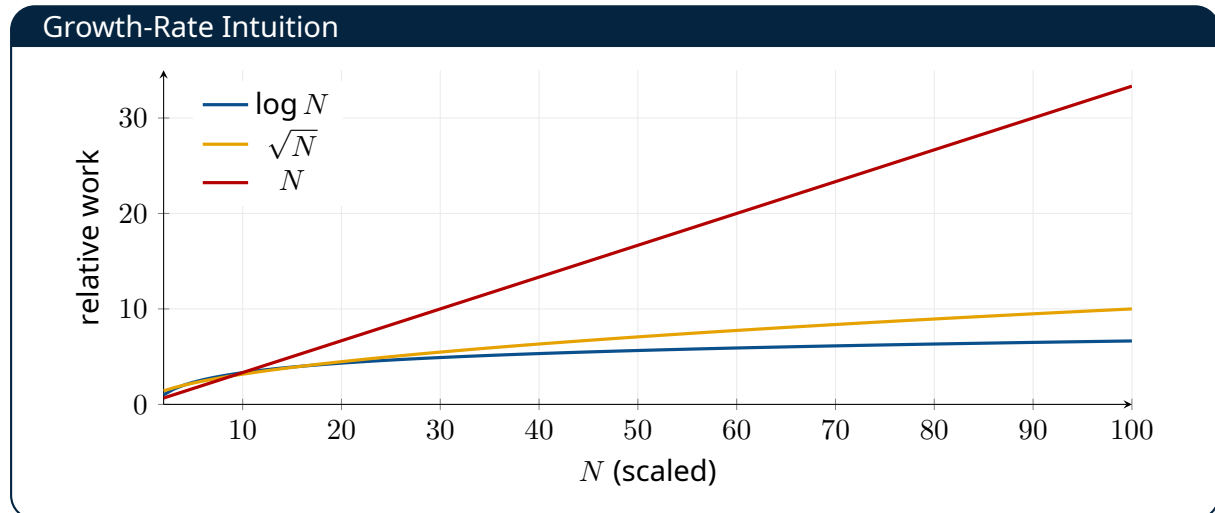
```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int MAXN = 1000005;
5 int spf[MAXN];
6
7 // 1. Precomputation: O(N log log N)
8 void sieve_spf() {
9     for (int i = 1; i < MAXN; i++) spf[i] = i; // Initialize SPF to
10    itself
11    for (int i = 2; i * i < MAXN; i++) {
12        if (spf[i] == i) { // i is prime
13            for (int j = i * i; j < MAXN; j += i) {
14                if (spf[j] == j) { // Only update if it hasn't been
15                    updated yet
16                        spf[j] = i;
17                    }
18                }
19            }
20        }
21    }
22
23    // 2. Querying prime factors: O(log X)
24    vector<int> getFactorization(int x) {
25        vector<int> ret;
26        while (x != 1) {
27            ret.push_back(spf[x]);
28            x = x / spf[x];
29        }
30    }
  
```

```
29     return ret;
30 }
31
32 int main() {
33     sieve_spf();
34     vector<int> factors = getFactorization(120);
35     // factors will be {2, 2, 2, 3, 5}
36     return 0;
37 }
```


7. Time Complexity Reference

Understanding the complexities of number theory algorithms helps you decide when to precompute versus when to calculate on the fly.

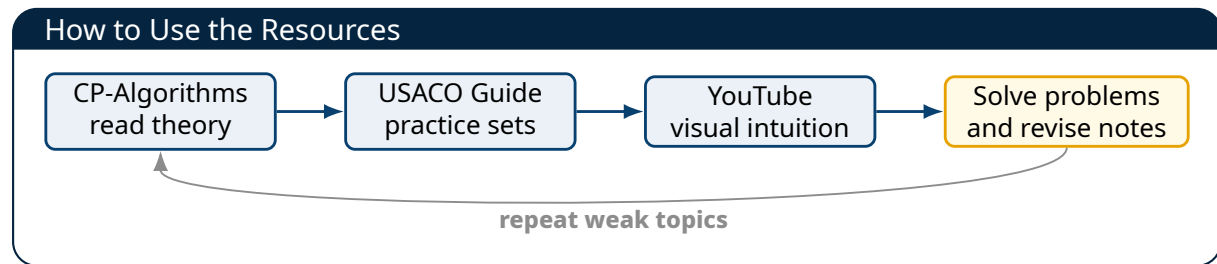


Algorithm	Time Complexity	Space Complexity
Binary Exponentiation	$O(\log B)$	$O(1)$
Euclidean Algorithm (GCD)	$O(\log(\min(A, B)))$	$O(1)$
Check if N is prime (Naive)	$O(\sqrt{N})$	$O(1)$
Prime Factorization (Naive)	$O(\sqrt{N})$	$O(1)$
Sieve of Eratosthenes	$O(N \log \log N)$ precomputation	$O(N)$
Fast Prime Factorization (SPF)	$O(\log X)$ per query	$O(N)$ for SPF array

Note: $\log \log N$ grows incredibly slowly. For $N = 10^9$, $\log \log N$ is practically a constant ≈ 5 . Therefore, $O(N \log \log N)$ behaves almost linearly like $O(N)$.

8. Recommended Resources

Number theory requires mathematical observation. For deeper dives and practice problems, utilize these curated resources:



- **CP-Algorithms:** The absolute gold standard for competitive programming math.
 - [Binary Exponentiation](#)
 - [Sieve of Eratosthenes](#)
- **USACO Guide:**
 - [Gold: Modular Arithmetic](#)
 - [Gold: Divisibility & Primes](#)
- **YouTube Channels:**
 - [Luv's Number Theory Playlist](#) (Detailed breakdown of Sieve, SPF, and modulo properties).
 - [Striver: Prime Maths for CP](#) (Excellent overview of Sieve complexities).